
Assignment 4 Report: Memory-Mapped Files

1 Introduction [2']

This assignment focuses on implementing the `mmap()` and `munmap()` system calls in OS xv6. The goal is to extend the virtual memory subsystem to support file-backed memory mapping, enabling user programs to map files directly into their address space. Unlike traditional file I/O using `read()` and `write()`, `mmap()` allows processes to access file contents through memory operations, while the kernel handles page faults and synchronizes data between memory and disk.

Our work introduces a new kernel data structure, the `vma` (Virtual Memory Area), to record metadata for each mapping, including its start address, length, protection flags, mapping type, associated file, and file offset. The system supports *lazy allocation*: physical pages are not allocated at mapping time but are created only when a page fault occurs during user access. This strategy improves memory efficiency and reflects the design used in modern OS.

We modified 4 kernel components: `sysfile.c`, `proc.c`, `trap.c`, and `proc.h`. The `sys_mmap()` and `sys_munmap()` functions were implemented to create and remove mappings; `trap.c` was extended to handle page faults and load file data into memory on demand; and `proc.c` was updated to copy VMAs during `fork()` and to release them in `exit()`. The final design achieves correct functionality for `mmap()` across multiple processes, passes all provided tests (including `fork_test`), and maintains compatibility with the existing xv6 virtual memory management framework.

Overall, this assignment provided hands-on experience with kernel-level memory management and clarified how operating systems implement efficient and consistent file-backed virtual memory.

2 Design [5']

The design of our `mmap()` implementation extends xv6's virtual memory management to support file-backed memory mapping with lazy allocation and proper synchronization. Our approach aims to remain consistent with xv6's minimalist philosophy while reproducing key behaviors of modern operating systems such as UNIX and Linux. This section describes the data structures, memory layout policy, and the interactions between key kernel modules.

2.1 Overall Architecture

The implementation introduces a new abstraction, the Virtual Memory Area (VMA), which describes each mapping created by `mmap()`. Each process maintains a fixed-size VMA table, `p->vma[VMASIZE]`, that records all active mappings. The system follows a lazy allocation model: pages are not allocated during `mmap()` itself but only when a page fault occurs. At that point, the kernel allocates a physical page, reads the corresponding data from the file, and establishes the mapping in the page table.

The design integrates changes across four kernel components:

- `proc.h`: defines the `vma` structure and adds it to the process structure.
- `sysfile.c`: implements `sys_mmap()` and `sys_munmap()` system calls.
- `trap.c`: extends the page fault handler to perform on-demand loading.
- `proc.c`: updates `fork()` and `exit()` to properly inherit and release mappings.

2.2 VMA Data Structure

Each VMA entry holds metadata about one mapping, including its start address, length, protection flags, file pointer, and file offset. We chose a fixed table size (16 entries per process) for simplicity and to match xv6's static resource model. This structure allows constant-time iteration when searching for the mapping that covers a faulting address. Unused entries are marked by `used = 0`. By storing a reference-counted file `* pointer`, the mapping can share the same underlying file object across processes.

```
#define VMASIZE 16

// ----- Virtual Memory Area (VMA) -----
// Each process keeps up to VMASIZE entries describing file-backed
// mappings created by mmap(). We record only metadata here; physical
// pages are allocated lazily on page fault in usertrap().
//
// Notes:
// - start/length are page-aligned virtual range.
// - prot/flags mirror the mmap() arguments (e.g., PROT_READ, MAP_SHARED).
// - file holds a referenced struct file (refcount bumped in sys_mmap).
// - offset is the file offset (page-aligned) for the start of the mapping.
struct vma {
    int used;           // slot is in use when set to 1
    uint64 start;       // start VA of mapping (page-aligned)
    uint64 length;      // length in bytes (page-aligned)
    int prot;           // PROT_READ/PROT_WRITE/PROT_EXEC
    int flags;          // MAP_SHARED / MAP_PRIVATE
    struct file *file;  // backing file (referenced)
    uint offset;        // starting file offset (page-aligned)
};
```

Figure 1: VMA Data Structure Design

2.3 Address Space Layout Policy

When a process calls `mmap()`, the kernel assigns a virtual address for the mapping. Our design uses a **bottom-up allocation policy**, which starts from the end of the user heap (`p->sz + 2 * PGSIZE`) and searches upward for the first non-overlapping region. This differs from the reference implementation's top-down placement near `MAXVA`. The bottom-up approach keeps mappings spatially close to the heap, avoids collisions with the trampoline and stack, and simplifies address management. All mappings are page-aligned using `PGROUNDUP()` and `PGROUNDDOWN()` to maintain correct alignment and reduce fragmentation.

2.4 Lazy Allocation and Page Fault Handling

Physical pages are allocated only when the mapped region is accessed. In `usertrap()`, the kernel handles page faults caused by load or store operations (`scause = 13` or `15`):

1. Identify the faulting virtual address using `r.stval()` and align it to a page boundary.
2. Search the process's VMA table for a matching region.
3. Allocate a physical page with `kalloc()` and zero-initialize it.
4. Read one page of data from the mapped file using `mapfile()` based on the offset.
5. Insert the new mapping into the page table via `mappages()` with permissions derived from `prot`.

This design ensures that memory and I/O are both demand-driven, avoiding unnecessary reads and conserving physical memory.

2.5 Unmapping and Write-back

When `munmap()` is invoked, the kernel removes the specified range of pages and optionally writes modified data back to disk. If the mapping was created with `MAP_SHARED | PROT_WRITE`, each

mapped page is written back using `writel()` within `begin_op()/end_op()` transactions to preserve file system consistency. For simplicity, the system conservatively writes all mapped pages instead of checking dirty bits. Partial unmapping is supported at the beginning or end of a region by adjusting the VMA boundaries; middle splitting is not implemented.

2.6 Process Fork and Exit Semantics

To ensure correct process semantics, the VMA table is duplicated during `fork()`: each child process copies the parent's VMA metadata but not the physical pages. File references are duplicated using `filedup()` to maintain shared access to the same underlying file. Both parent and child handle page faults independently after the fork. During `exit()`, all remaining VMAs are explicitly unmapped and closed before freeing the page table. This cleanup avoids residual leaf PTEs that could otherwise trigger a freewalk: leaf panic.

3 Environment and Execution [2']

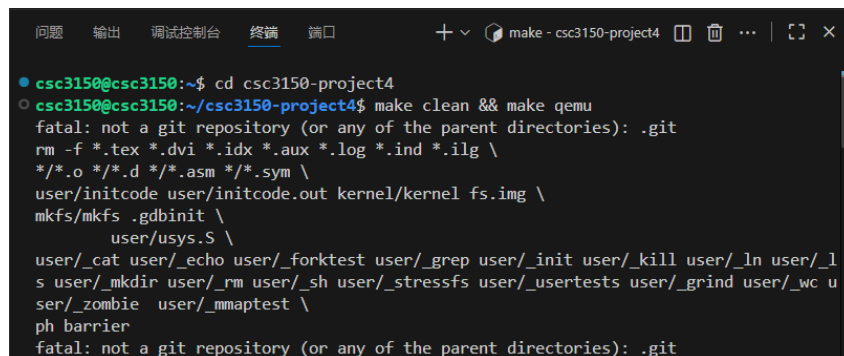
3.1 Environment Setup

The implementation and testing of this assignment were conducted in the environment provided by the course, using the folder `csc3150-project4` distributed by the teaching team. Students are required to use this project directory as the working base, which already contains the xv6 source code, QEMU emulator configuration, and testing scripts.

To integrate the implemented functions, simply replace the original four source files in the `kernel/` directory with the modified versions provided in the student's submission package:

- `proc.c`
- `proc.h`
- `sysfile.c`
- `trap.c`

These files contain all the necessary modifications to support the `mmap()` and `munmap()` system calls, lazy page allocation, and VMA management.



```
问题 输出 调试控制台 终端 窗口 + v make - csc3150-project4
csc3150@csc3150:~$ cd csc3150-project4
csc3150@csc3150:~/csc3150-project4$ make clean && make qemu
fatal: not a git repository (or any of the parent directories): .git
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
  */*.o */*.d */*.asm */*.sym \
  user/initcode user/initcode.out kernel/kernel fs.img \
  mkfs/mkfs .gdbinit \
  user/usys.S \
  user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls
  user/_mkdir user/_rm user/_sh user/_stressfs user/_usertests user/_grind user/_wc u
  ser/_zombie user/_mmaptest \
  ph barrier
fatal: not a git repository (or any of the parent directories): .git
```

Figure 2: Development Environment Setup (CSC3150 Project 4)

3.2 Execution Procedure

After replacing the files, compile and run xv6 using the following commands inside the project root directory:

```
$ make clean && make qemu
```

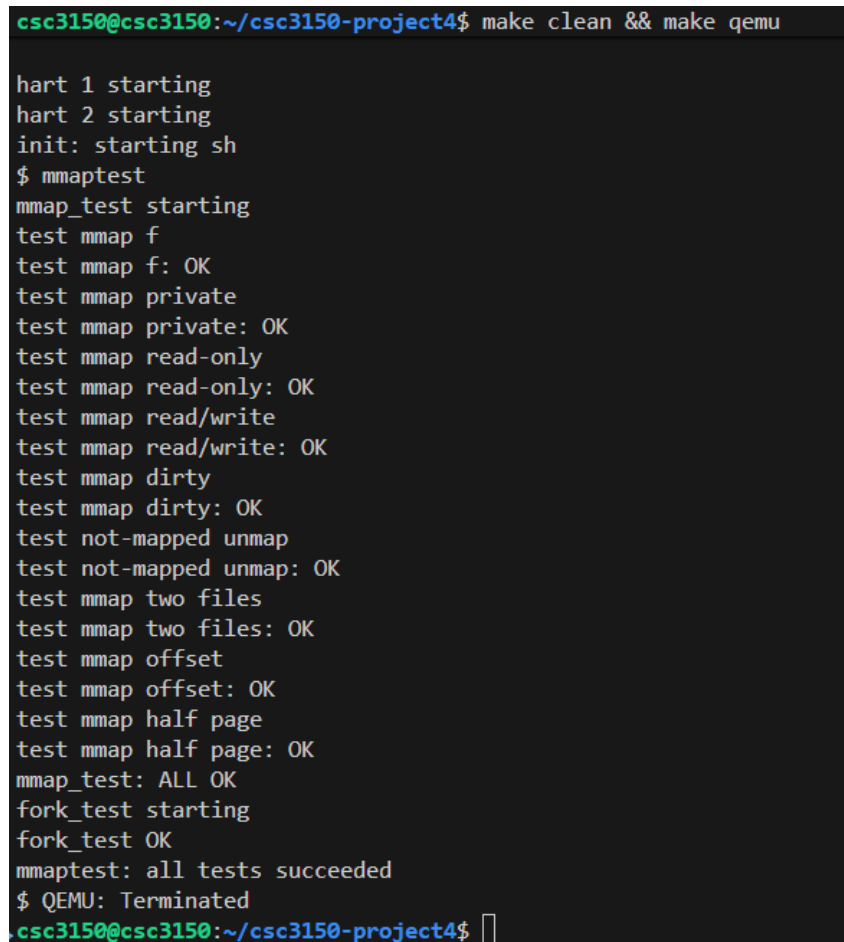
Once xv6 boots in the QEMU emulator, run the provided user-level test program `mmaptest` to verify the correctness of the implementation:

```
$ mmaptest
```

The program automatically tests the core functionalities, including:

- basic `mmap()` creation and data access,
- shared and private mapping consistency,
- process inheritance through `fork()`,
- and proper unmapping behavior via `munmap()`.

A successful implementation should pass all test cases, including `fork_test`, without any kernel panic or page fault outside VMA.



```
csc3150@csc3150:~/csc3150-project4$ make clean && make qemu
hart 1 starting
hart 2 starting
init: starting sh
$ mmaptest
mmap_test starting
test mmap f
test mmap f: OK
test mmap private
test mmap private: OK
test mmap read-only
test mmap read-only: OK
test mmap read/write
test mmap read/write: OK
test mmap dirty
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test mmap two files
test mmap two files: OK
test mmap offset
test mmap offset: OK
test mmap half page
test mmap half page: OK
mmap_test: ALL OK
fork_test starting
fork_test OK
mmaptest: all tests succeeded
$ QEMU: Terminated
csc3150@csc3150:~/csc3150-project4$
```

Figure 3: Execution of `mmaptest` in QEMU showing successful test results

In summary, students only need to compile, run the emulator, and execute the `mmaptest` program to confirm that the kernel correctly implements file-backed memory mapping, lazy page loading, and safe cleanup during process termination.

4 Conclusion [2']

In this assignment, we successfully extended the xv6 operating system to support file-backed memory mapping through the implementation of the `mmap()` and `munmap()` system calls. The project involved designing a new kernel data structure, the Virtual Memory Area (VMA), and integrating it with xv6's existing process and memory management framework. Our design realized a lazy page

allocation mechanism that loads pages into memory only when a page fault occurs, ensuring efficient memory usage and realistic file I/O behavior.

Throughout this work, we made coordinated modifications to several key kernel components, including `sysfile.c`, `trap.c`, and `proc.c`. These changes collectively enabled the kernel to create, maintain, and clean up file mappings while preserving system stability and compatibility with xv6's minimalist architecture. The system was validated using the provided `mmaptest` program, which confirmed correct functionality for shared mappings, process inheritance, and page-fault-driven on-demand loading.

Through completing this assignment, we gained a deeper understanding of virtual memory management in operating systems—specifically how page tables, file systems, and process structures interact to provide a unified address space abstraction. We also learned the importance of synchronization, resource cleanup, and correctness in kernel-level design. Overall, the project bridged theoretical concepts of memory management with practical kernel programming experience, reinforcing our grasp of how modern operating systems efficiently manage memory-mapped files.